PHASE 3

Project Design

Script-Controlled ACL – Restrict Record Access Based on Field Value

Field	Details
Date	30.9.2026
Team ID	6ab4dcafa238dc999eb5d928
Project Name	Script-Controlled ACL – Restrict Record Access Based on Field Value
Platform	ServiceNow (Global scope)
Custom Table	Institution Details (u_institution_details)

Purpose

This phase defines the design of the solution: the data model, users and roles, the Access Control List (ACL) design for Read, Create, Write and Delete operations, the script-controlled access logic, the test design, and the supporting implementation evidence for restricting record access based on a field value in ServiceNow.

1. Design Overview

Institution records (student, faculty, branch, contact details) are stored in a custom table. These records should not be visible to everyone. The design uses a script-controlled ACL so that only users who hold the required role, and only for records where the **Branch** field is **EEE**, can view them. Administrators retain full access.

Design goals

- Store institution data in a dedicated custom table (u_institution_details).
- Restrict record visibility using a Read ACL with a role, a data condition (Branch is EEE) and an advanced script.
- Control create, edit and delete separately through dedicated roles (bb2, bb3, bb4).
- Keep full access for the admin role through the Admin overrides setting.
- Verify the design by impersonating users with different role combinations.

2. Design Flow

Admin Login → User & Roles (bb1–bb4) → Table u_institution_details → Sample Records (ECE, EEE, CSE) → Elevate security_admin → Read ACL → Create ACL → Write ACL → Delete ACL → Impersonate & Verify

3. Data Design

Table: **Institution Details** — Name: **u_institution_details** — Extends table: None (standalone).

Field	Type	Purpose
Student Roll Number	Auto Number	Unique record number (INS0001001 format).
Student Name	Reference – User	Student linked to a user record.
Faculty Name	Reference – User	Faculty member linked to a user record.
Branch	Choice (ECE, EEE, CSE)	**Key field used by the ACL data condition.**
Email	String	Contact email.
Phone Number	String	Contact phone number.
Description	Multi-line String	Notes; also used to test the Write ACL.

4. Users and Roles Design

A test user **EEE User** (eeeuser@gmail.com) is created and assigned all four custom roles. Each role controls one operation, so permissions can be added step by step during testing.

Role	Controls	Used by ACL
bb1	View records (Read)	Read ACL – u_institution_details
bb2	Add new records (Create)	Create ACL – u_institution_details
bb3	Edit records (Write)	Write ACL – u_institution_details
bb4	Remove records (Delete)	Delete ACL – u_institution_details
admin	Full access to all records	Admin overrides = true on every ACL

5. ACL Design Matrix

All ACLs are of Type **record**, Decision Type **Allow If**, Active **true**, on table **u_institution_details**. The security_admin role must be elevated before creating them.

Operation	Requires role	Data condition	Advanced script	Effect
read	bb1	Branch is EEE	Yes (checks bb1)	Only EEE records are visible.
create	bb2	None	No	New button becomes available.
write	bb3	None	No	Fields can be edited (inline/form).
delete	bb4	None	No	Records can be deleted.

6. Access Decision Logic

For every read request, ServiceNow evaluates the Read ACL. Access is granted only when all of the following are true:

1. The user has the **bb1** role (or is an admin, because Admin overrides is enabled).
2. The record matches the data condition **Branch is EEE**.
3. The advanced script returns **true**.

If any check fails, the record is hidden. A user without the role sees an empty list, and a user with the role never sees ECE or CSE records.

7. Script Design (Read ACL)

The script below is placed in the Script field of the Read ACL (Advanced = true). The Branch filtering is done by the data condition; the script confirms the role.

```
(function () {
    if (gs.hasRole('bb1')) {
        return true;
    } else {
        return false;
    }
})();
```

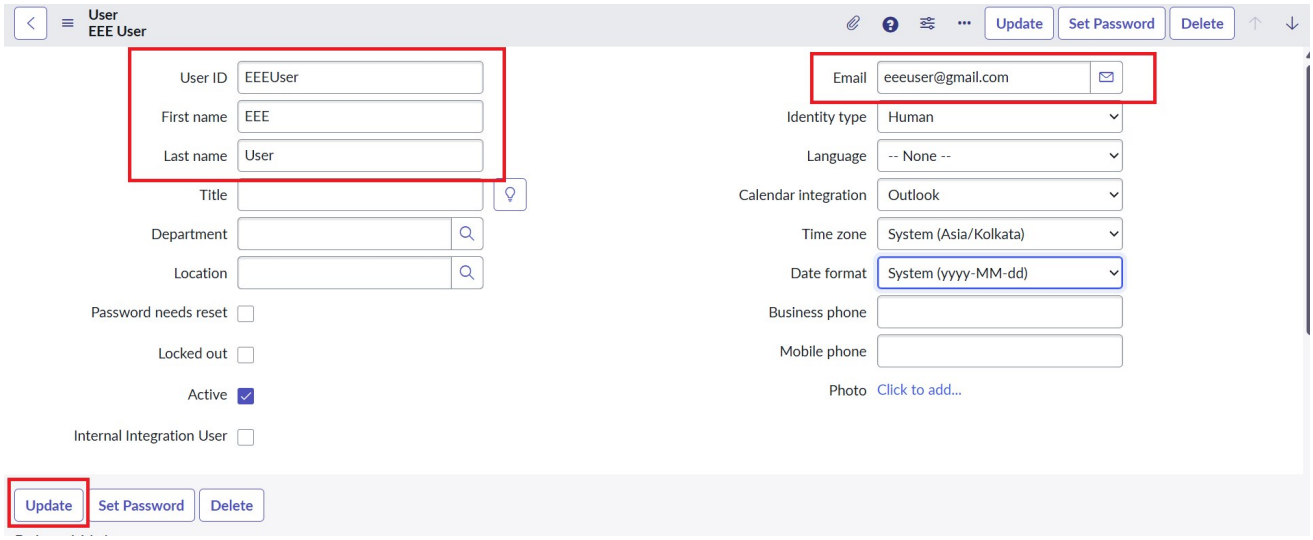
8. Test Design

Scenario	Roles / User	Expected result
Read	EEE User with bb1	Only EEE branch records are listed.
Read (no role)	User without bb1	No records to display.
Read (admin)	Admin	All records (ECE, EEE, CSE) are listed.
Create	bb1 + bb2	EEE records visible and the New button is shown.
Write	bb1 + bb2 + bb3	EEE records can be edited (Description updated).
Delete	bb1 + bb2 + bb3 + bb4	Selected EEE records can be deleted.

9. Implementation Evidence

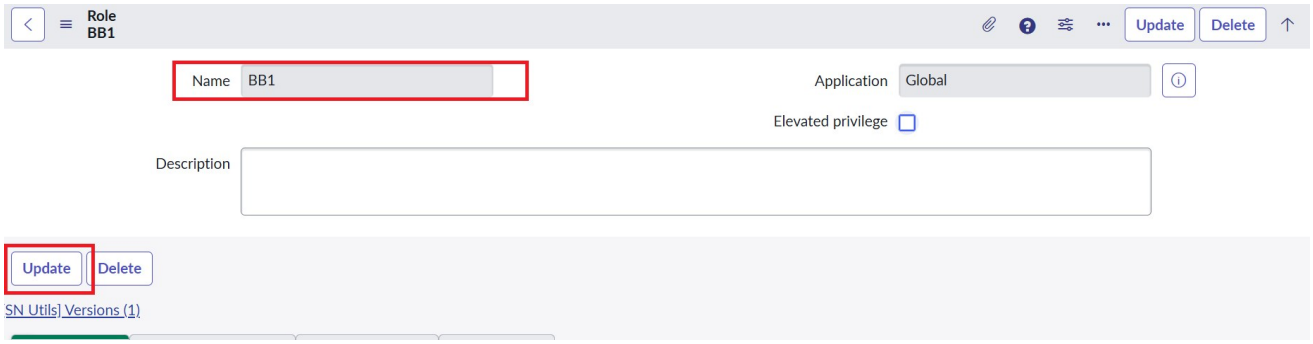
The screenshots below are arranged in workflow order. Only the screenshots needed to explain the design and verification are included.

Screenshot 1 — User creation



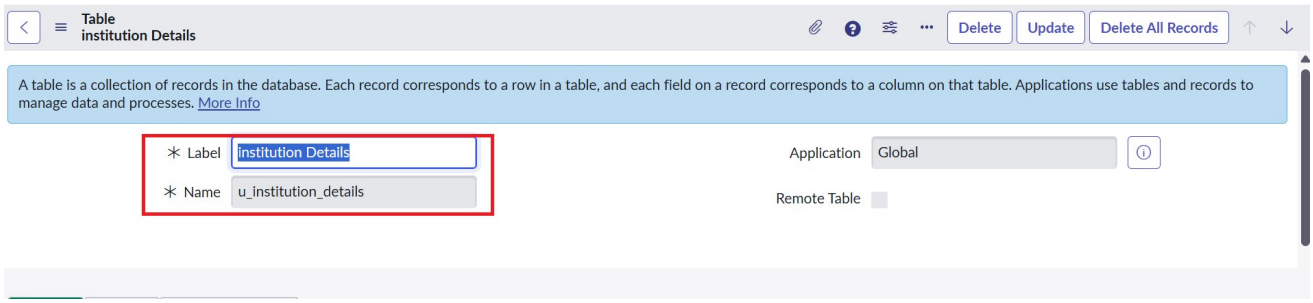
EEE User created with email eeeuser@gmail.com

Screenshot 2 — Role creation



Custom role BB1 (BB2, BB3 and BB4 are created the same way)

Screenshot 3 — Create Institution Details table



Custom table u_institution_details

Screenshot 4 — Table columns

Table Columns for text Search 1 to 13 of 13

Dictionary Entries

Column label	Type	Reference	Max length	Default value
Updates	Integer	(empty)	40	
Created by	String	(empty)	40	
Student Roll Number	String	(empty)	40	javascript:global.getNextObjNumberPadded();
Description	String	(empty)	1,000	
Updated by	String	(empty)	40	
Phone Number	String	(empty)	40	
Sys ID	Sys ID (GUID)	(empty)	32	
Created	Date/Time	(empty)	40	
Faculty name	Reference	User	32	
Branch	String	(empty)	40	
Updated	Date/Time	(empty)	40	
Student Name	Reference	User	32	
Email	String	(empty)	40	
Insert a new row...				

Fields added to the table

Screenshot 5 — Sample records

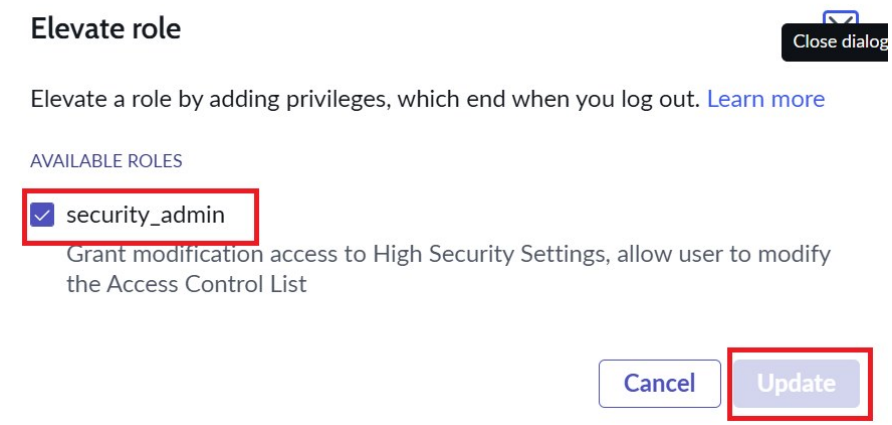
institution Details Student Roll Numbe Search Actions on selected rows...

All

Student Roll Number	Faculty name	Student Name	Branch	Email	Phone Number
INS0001006	ITIL User	Praneeth	ECE	praneeth@gmail.com	8989090906
INS0001008	Malla Sravan kumar	Abraham Lincoln	CSE	sravan@gmail.com	8978908989
INS0001005	ITIL User	Srinu Nandipalli	ECE	srinu@gmail.com	9089909090
INS0001007	Abel Tuter	Guna Sai	EEE	saiguna@gmail.com	9090898765
INS0001001	Abraham Lincoln	Yaswanth Nikku	CSE	yaswanth@gmail.com	8989898989
INS0001004	Abel Tuter	Prashanth Reddy	EEE	prashanth@gmail.com	9090909090

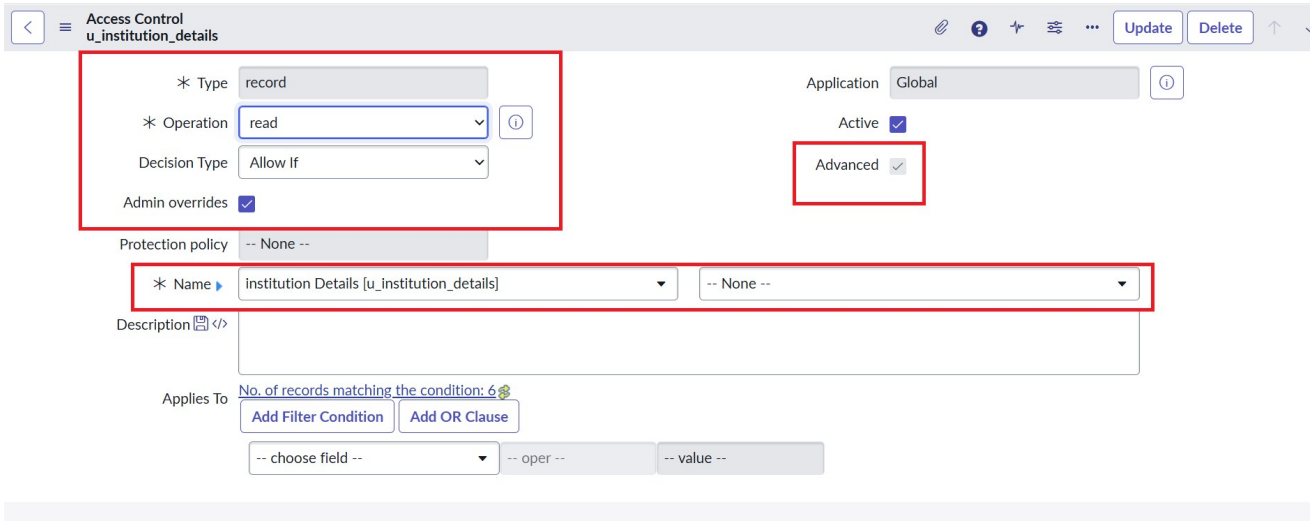
Records with different Branch values (ECE, EEE, CSE)

Screenshot 6 — Elevate security_admin role



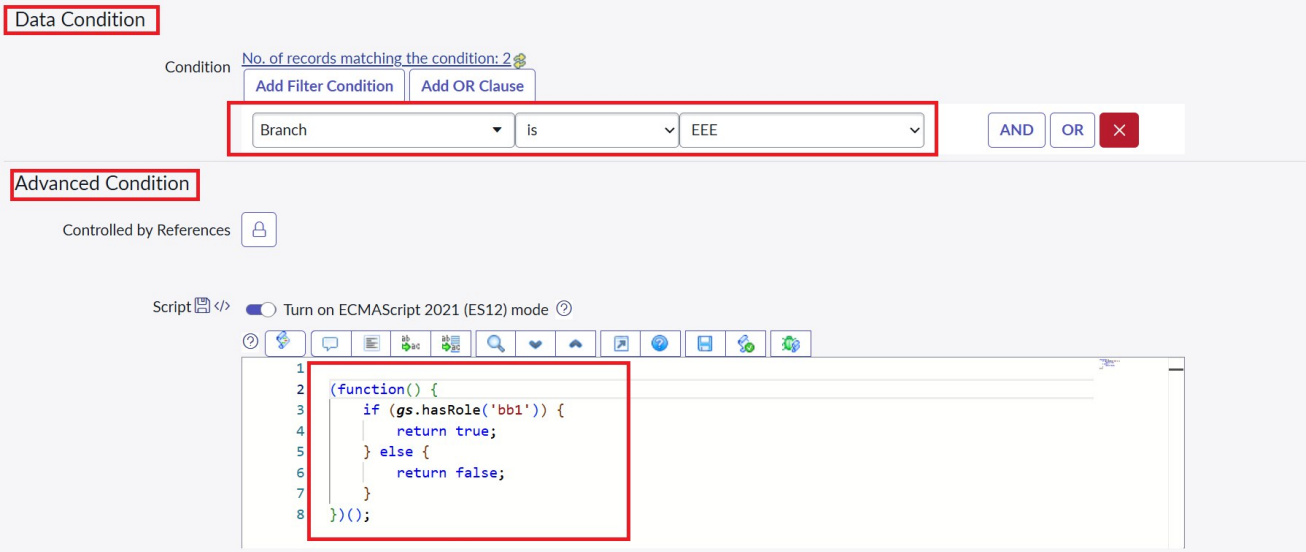
Elevated role required to create ACLs

Screenshot 7 — Read ACL configuration



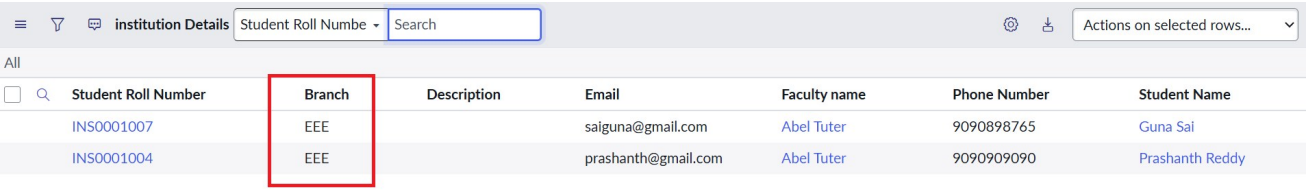
Type: record, Operation: read, Advanced: true

Screenshot 8 — Data condition and script



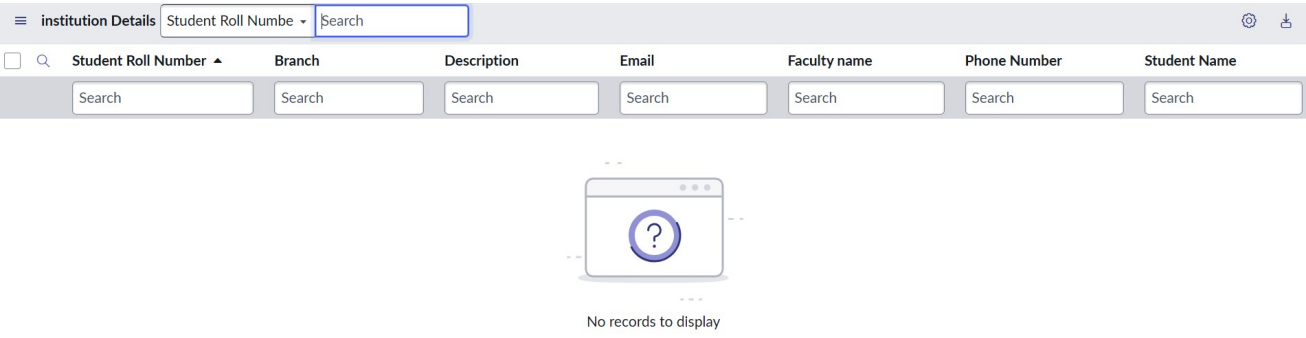
Branch is EEE with the advanced role-check script

Screenshot 9 — Read ACL result – bb1 user



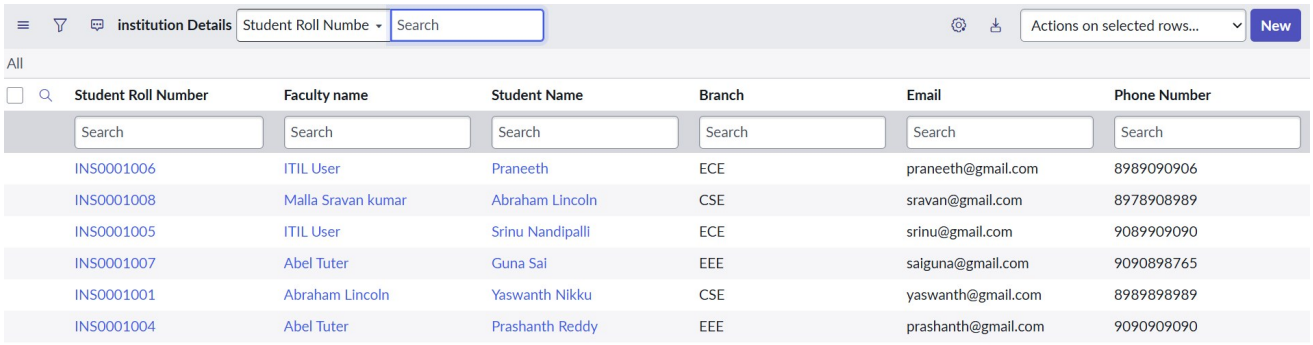
User with the bb1 role sees only EEE branch records

Screenshot 10 — Read ACL result – user without role



No records to display

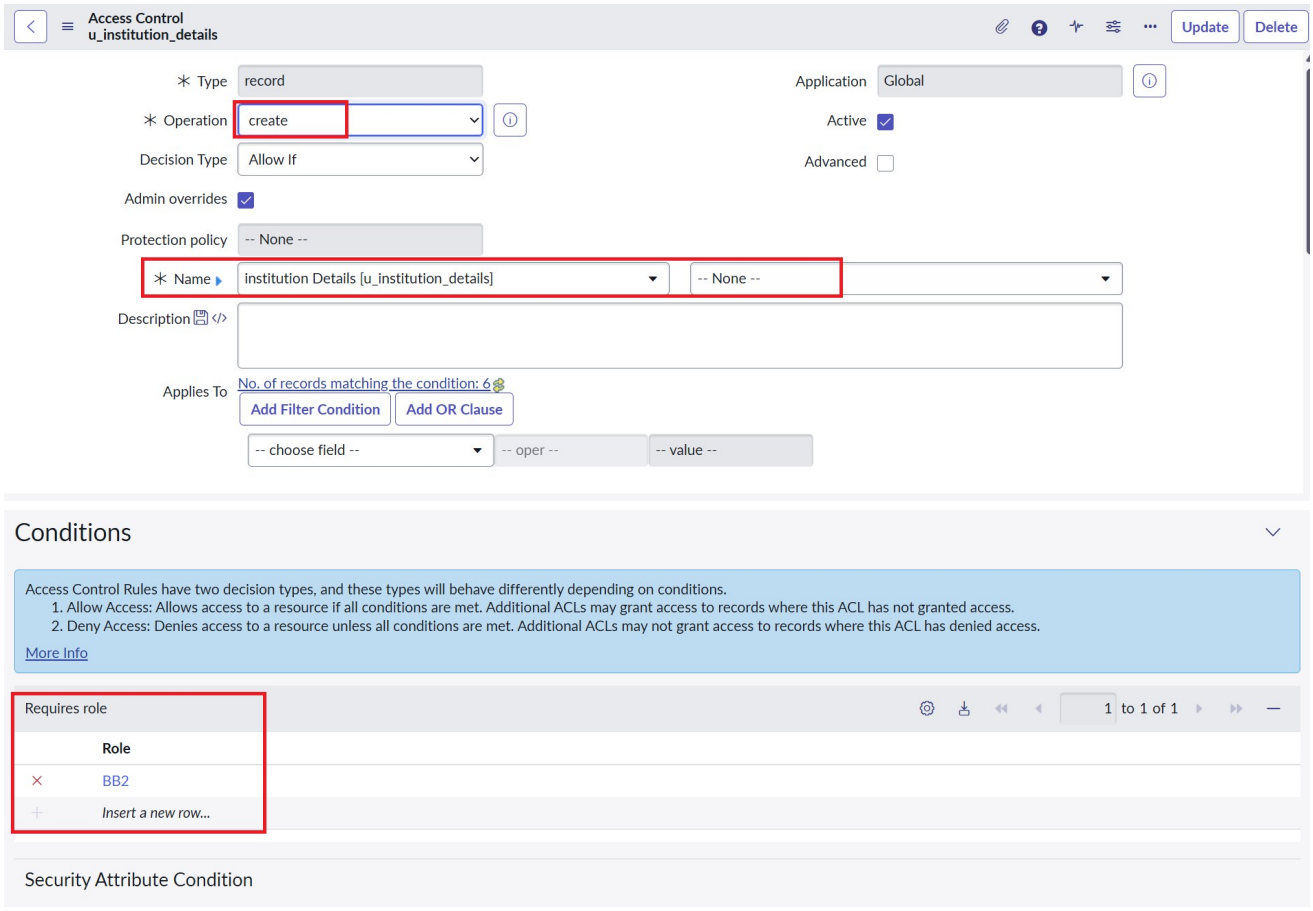
Screenshot 11 — Read ACL result – admin



Student Roll Number	Faculty name	Student Name	Branch	Email	Phone Number
INS0001006	ITIL User	Praneeth	ECE	praneeth@gmail.com	8989090906
INS0001008	Malla Sravan kumar	Abraham Lincoln	CSE	sravan@gmail.com	8978908989
INS0001005	ITIL User	Srinu Nandipalli	ECE	srinu@gmail.com	9089909090
INS0001007	Abel Tuter	Guna Sai	EEE	saiguna@gmail.com	9090898765
INS0001001	Abraham Lincoln	Yaswanth Nikku	CSE	yaswanth@gmail.com	8989898989
INS0001004	Abel Tuter	Prashanth Reddy	EEE	prashanth@gmail.com	9090909090

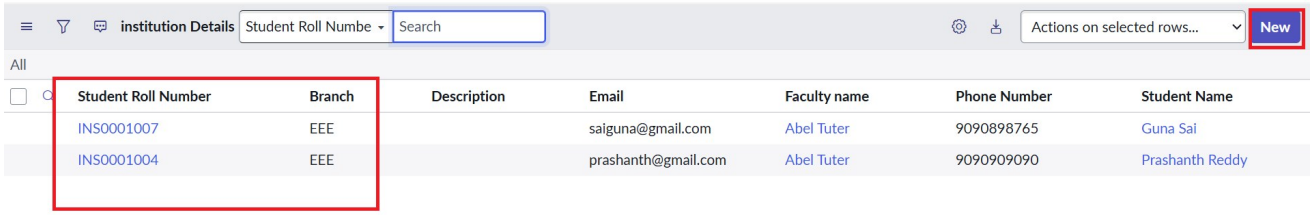
Admin sees all records regardless of branch

Screenshot 12 — Create ACL configuration



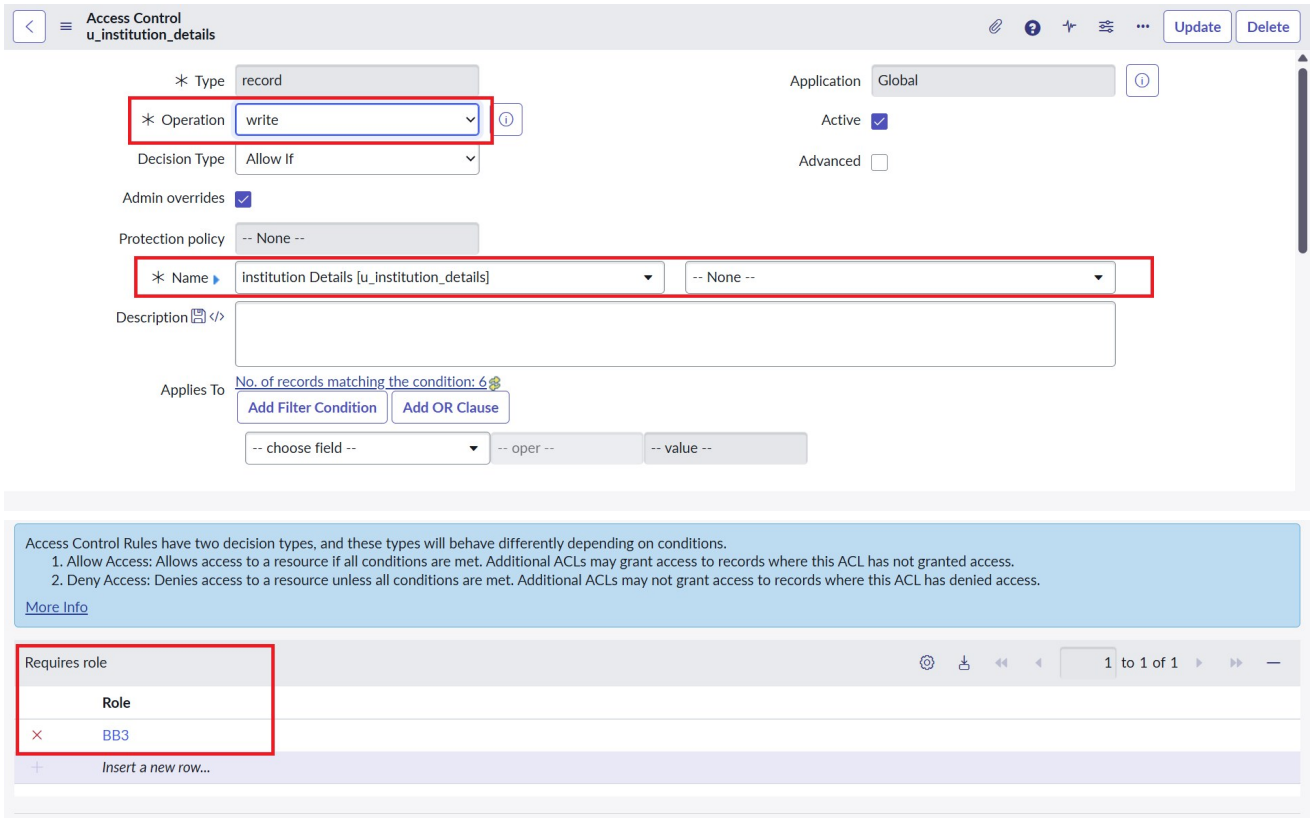
Operation: create, Requires role: BB2

Screenshot 13 — Create ACL result



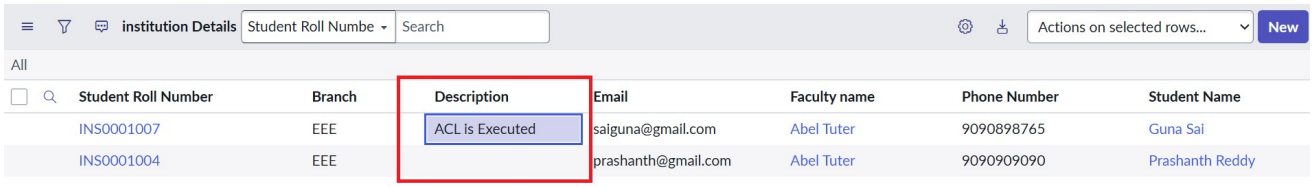
With bb1 and bb2, the New button appears

Screenshot 14 — Write ACL configuration



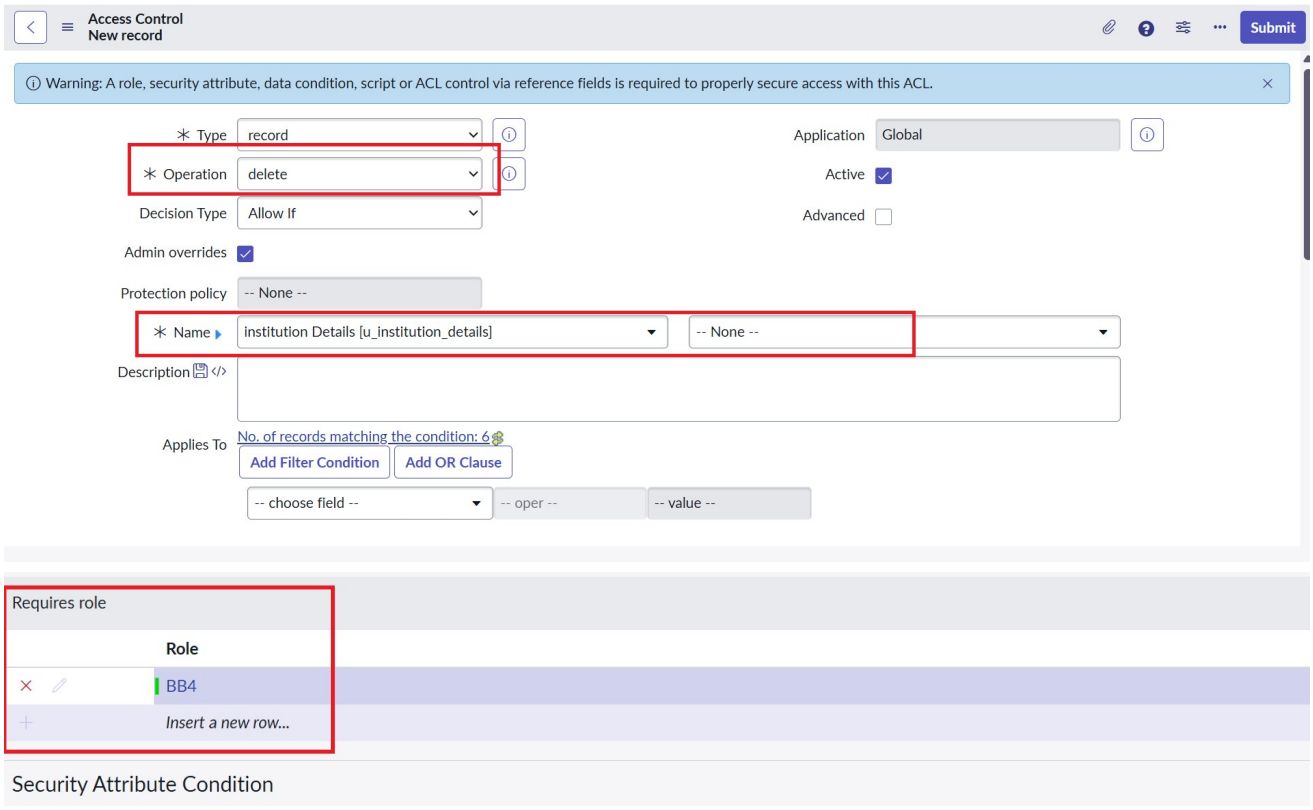
Operation: write, Requires role: BB3

Screenshot 15 — Write ACL result



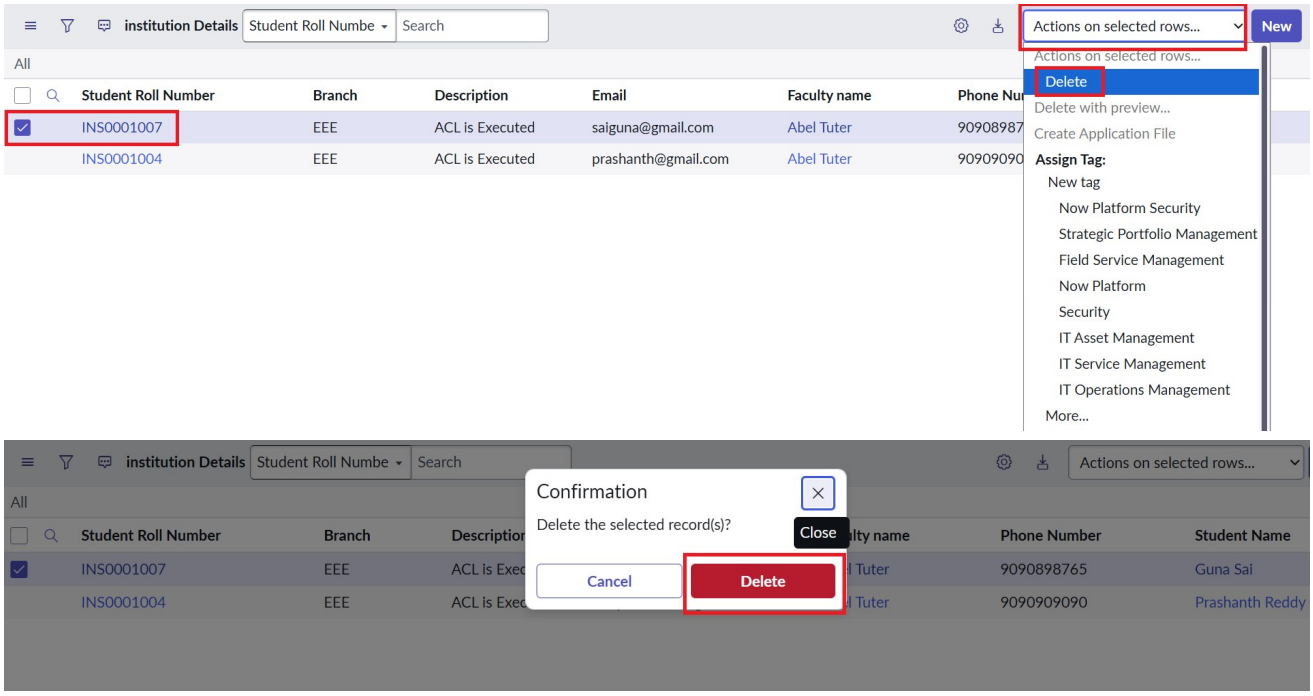
With bb1, bb2 and bb3, the Description field is editable

Screenshot 16 — Delete ACL configuration



Operation: delete, Requires role: BB4

Screenshot 17 — Delete ACL result



With bb1 to bb4, selected EEE records can be deleted

10. Expected Outcome

The design delivers record-level security for the Institution Details table. Users with bb1 see only EEE branch records, users without the role see nothing, and admins see everything. The Create, Write and Delete ACLs add separate control over adding, editing and removing records. Together, they show how roles, field values and scripts are evaluated to protect data from unauthorized viewing, modification, creation and deletion.